KEEPER MEMORY

Solidity Security Copilot with Persistent Memory

ETHGlobal Open Agents Hackathon • Avril – Mai 2026

Marius · Cyriac · Nohem | CNM Agency

Stack	Sponsors	Paiements	Ancrage
Python · Next.js · Solidity	KeeperHub · 0G · ENS	x402 (USDC)	KeeperHub MCP

1. Vision & Problème

Le problème

Le marché de la sécurité Solidity est binaire : soit aucun outil sérieux, soit un audit professionnel à 50 000–200 000 $. Entre les deux, rien.

Les outils existants (Slither, Mythril) sont stateless : chaque analyse repart de zéro, sans mémoire des patterns découverts sur d'autres contrats. Les faux positifs sont massifs et non filtrés par le contexte historique.

> **Insight clé**
> Un audit humain est précieux parce que l'auditeur se souvient : il reconnaît un pattern qu'il a vu 6 mois plus tôt sur un autre protocole. Aucun outil stateless ne peut faire ça.

La solution : Keeper Memory

Un copilot Solidity avec mémoire persistante et vérifiable. Un agent qui analyse continuellement des contrats, apprend de chaque analyse via une mémoire tridimensionnelle (Cognee), se paie via x402 (USDC, sans API key), et ancre chaque finding critique onchain via KeeperHub.

Outil	Mémoire	Paiement	Limite
Slither / Mythril	Aucune	Gratuit	Faux positifs non filtrés
GPT-4 / Claude direct	Contexte conversation	Abonnement	Reset à chaque session
Keeper Memory	Tridimensionnelle (Cognee)	x402 USDC pay-per-use	—

2. Onboarding — CLI Tool

Keeper Memory s'installe et s'utilise exactement comme Slither. Global sur la machine, appelé depuis n'importe quel dossier Solidity — pas une dépendance de projet.

Installation

```
pip install keeper-memory
```

Dépendances installées automatiquement : Slither, Cognee, x402-py, MCP client. Rien d'autre à installer manuellement.

Setup initial — une seule fois

```
keeper-memory init
# → Generating local wallet...
# ✓ Address: 0xYourWallet (Base Sepolia)
# → Fund your wallet with USDC to pay for audits
# ✓ Config saved to ~/.keeper-memory/config.json
# ✓ Memory stored in ~/.keeper-memory/memory/
```

Le wallet est généré localement, chiffré AES avec un mot de passe. Pas besoin de Metamask. La mémoire Cognee est globale par machine et persiste entre tous les projets.

Usage quotidien

```
cd mon-projet-defi/

keeper-memory audit ./src/                    # répertoire
keeper-memory audit ./src/Vault.sol           # fichier spécifique
keeper-memory audit 0xUniswapV2Router02       # adresse onchain
```

Output terminal

```
Keeper Memory v0.1.0
____________________________________________________
[x402]    Paying 1.00 USDC for audit...   ✓ tx: 0x8fa3...
[Phase 0] Resolving...          ✓ 4 files (Vault.sol, Router.sol...)
[Phase 1] Inventory...          ✓ 2 flags (delegatecall, unchecked)
[Phase 2] Slither...            ✓ 7 findings (2 HIGH, 3 MED, 2 LOW)
[Phase 3] Triage...             risk_score: 7.8 / 10
[Phase 4] Investigating...
  > read_contract(Vault.sol, withdraw)
  > query_memory('reentrancy external call')
  Memory hit: Euler Finance hack (2024-03-15) — $197M
  Finding CONFIRMED — anchoring onchain now...
  ✓ f-001 anchored → 0x7f2e... (KeeperHub)
  > read_contract(Router.sol, setFeeRecipient)
```

```
  Finding CONFIRMED — missing onlyOwner
  ✓ f-002 anchored → 0x3bc4... (KeeperHub)
[Phase 5] Verifying anchors... ✓ 2/2 confirmed
──────────────────────────────────────────────────────
VERDICT: HIGH RISK (7.8/10)
[HIGH] Vault.sol:142  Reentrancy — withdraw() external before state
[HIGH] Router.sol:89  Missing access control — setFeeRecipient()
Report:       ./keeper-memory-report.json
Onchain proof: https://etherscan.io/tx/0x3bc4...
```

Mémoire — propriété et portée

Aspect	Comportement
Portée	Globale par machine — persiste entre tous les projets Solidity
Propriété	Locale et privée — aucun audit partagé sans consentement
Croissance	Plus tu audites, plus l'agent reconnaît des patterns similaires
Roadmap 0G	Mémoire partagée opt-in via stockage décentralisé (post-hackathon)

Demo script — moment 'wow' pour les juges

Scénario en 2 audits

1. Auditer 0xEulerFinanceVault → finding reentrancy mémorisé dans Cognee, anchoré onchain via KeeperHub.2. Auditer un nouveau contrat inconnu avec pattern similaire.3. L'agent dit dans le terminal : "Memory hit: similar reentrancy — Euler Finance ($197M hack, March 2024) — confidence upgraded SUSPECTED → CONFIRMED" puis ancre immédiatement.L'agent a reconnu le pattern ET pris une action onchain de façon autonome. C'est le thème Open Agents.

3. Architecture Système

Couche	Technologie	Rôle
CLI	Python / Click	Entry point — commandes keeper-memory
Backend API	Python / FastAPI	Routes HTTP protégées par x402
Pipeline	Python / Anthropic SDK	6 phases d'analyse
Paiements audits	x402 USDC	Pay-per-audit — débloque mémoire collective
Mémoire	Cognee	Vectoriel + graphe + relationnel — locale
Storage patterns	0G Storage KV	Patterns anonymisés — kv_set/kv_get par pattern_hash
Ancrage hash	KeeperHub + AnchorRegistry.sol	rootHash 0G anchoré onchain — lien immuable
Identité agent	ENS — keeper-memory.eth	Identité lisible de l'agent + subnames auditeurs
Gas operator	Para MPC Wallet (KeeperHub)	Auto-provisionné — zéro gestion

Rôles distincts — KeeperHub vs 0G

	KeeperHub	0G Storage
Analogie	Notaire	Coffre-fort décentralisé
Ce qu'il fait	Exécute la tx onchain	Stocke le fichier/JSON de façon permanente
Ce qu'il contient	pattern_hash + rootHash_0G (32+32 bytes)	Pattern complet : snippet + description + fix
Accès	Public — Etherscan	Public — rootHash suffit pour récupérer
Si on disparaît	Hash vérifiable à jamais	Contenu accessible à jamais

Structure du projet Python

```
keeper_memory/
├── cli.py                  # entry point — @click commands
├── server.py               # FastAPI — routes x402
├── pipeline/
│   ├── phase0_resolve.py   # adresse/fichier → ResolvedContract + upstream
detection
│   ├── phase1_inventory.py # parse structurel + load known-findings
│   ├── phase2_slither.py   # analyse statique — diff only si fork détecté
│   ├── phase3_triage.py    # map/reduce claude-haiku
│   ├── phase4_agent.py     # agent adversarial claude-sonnet + 7 outils
│   └── phase5_anchor.py    # filet de sécurité — Direct Execution API
```

```
├── memory/
│   ├── cognee_client.py     # wrapper Cognee
│   └── known_findings.py    # déduplication — cross-ref avant chaque
finding
├── payments/
│   ├── x402_client.py       # CLI → paie le backend
│   └── x402_server.py       # Backend → vérifie paiement
├── storage/
│   └── zero_g_client.py     # 0G Storage KV — kv_set/kv_get patterns
├── ens/
│   └── subname_manager.py   # Namespace SDK — mint + update reputation
subnames
├── keeper/
│   ├── direct_api.py        # Direct Execution API (Option A)
│   └── mcp_tools.py         # MCP tools exposés à l'agent (Option B)
└── 0g/
    ├── 0g_upload.js         # wrapper Node — upload JSON → rootHash
    └── 0g_download.js       # wrapper Node — rootHash → JSON
```

4. Pipeline Technique — 6 Phases

Architecture inspirée du système d'audit interne de MockSystems (infrastructure blockchain) — notamment le pattern de déduplication via known-findings et la stratégie diff vs upstream.

Phase	Nom	Modèle	Description
0	Resolve	—	Adresse Ethereum ou fichier .sol → ResolvedContract. Détecte les forks connus (Uniswap, Aave...) pour restreindre le scope au delta uniquement.
1	Inventory	—	Parse structurel sans LLM. Charge les known-findings depuis la mémoire Cognee pour déduplication.
2	Slither	—	Analyse statique — sortie JSON filtrée. Sur un fork détecté, n'analyse que les fichiers du diff.
3	Triage	claude-haiku-4-5	Map/Reduce par fichier — risk_score 0-10, gate < 3.
4	Investigation	claude-sonnet-4-6	Agent adversarial + 7 outils — 30 turns max. Mindset : "comment un attaquant abuserait ce code ?"
5	Anchor (filet)	—	Direct Execution API — vérifie et complète les anchors manqués.
6	Report	—	JSON / PDF — findings avec prior audit reference + code sketch de fix.

Phase 0 — Détection fork upstream

Si le contrat audité est un fork d'un projet connu (Uniswap, Aave, Compound, OpenZeppelin...), on détecte l'upstream via les imports et on restreint le scope au diff uniquement. Les vulnérabilités dans le code upstream sont déjà connues et auditées — ce qui compte c'est ce que ce projet a modifié.

```
# phase0_resolve.py
KNOWN_UPSTREAMS = {
    'IUniswapV2Router': ('Uniswap/v2-core', 'v1.0.1'),
    'AaveV3Pool':       ('aave/aave-v3-core', 'v1.19.3'),
    'ERC4626':          ('OpenZeppelin/openzeppelin-contracts', 'v5.0.0'),
}

def detect_upstream(files: list[str]) -> Optional[UpstreamRef]:
    """Détecte l'upstream via les imports. Si trouvé → diff only."""
    for file in files:
        for marker, upstream in KNOWN_UPSTREAMS.items():
            if marker in open(file).read():
                return UpstreamRef(*upstream)
    return None  # pas de fork détecté → scan complet
```

Sur un fork détecté, le scope est réduit de ~95% — seuls les fichiers modifiés par rapport à l'upstream sont analysés. Gain direct en tokens et en précision.

Phase 1 — Déduplication via known-findings

Avant toute analyse, l'agent charge la liste des findings déjà documentés depuis Cognee. Chaque nouveau finding est cross-référencé avant d'être inclus dans le rapport — aucune vulnérabilité n'est reportée deux fois sur le même contrat.

```
# phase1_inventory.py
async def load_known_findings(contract_id: str) -> list[KnownFinding]:
    """Charge les findings déjà documentés pour ce contrat."""
    return await cognee.search(
        f'known findings for {contract_id}',
        scope='local',
        filter={'contract_id': contract_id, 'status': 'documented'}
    )

def is_duplicate(finding: Finding, known: list[KnownFinding]) -> bool:
    """Cross-référence via pattern_hash + location."""
    return any(
        k.pattern_hash == finding.pattern_hash
        for k in known
    )
```

Phase 4 — Mindset adversarial

Inspiré directement de la consigne de sécurité MockSystems : l'agent lit le code de façon adversariale AVANT de consulter sa checklist. La checklist est un filet de couverture, pas un point de départ.

> **AGENT_SYSTEM_PROMPT — extrait clé**
> Read code adversarially before consulting any checklist. Ask: how could an attacker abuse this? What assumptions does this code make that could be violated? What happens if the contract owner is malicious, unavailable, or acts unexpectedly?After adversarial review, use the checklist as a coverage backstop — verify each category was at least considered. The checklist is a floor, not a limit.

Phase 4 — Outils de l'agent (7 outils)

Outil	Type	Description
read_contract(file, fn)	Lecture	Lit le source Solidity d'une fonction
search_pattern(regex)	Lecture	Recherche regex sur tous les .sol
get_call_graph(contract)	Lecture	Call graph Slither
get_storage_layout(contract)	Lecture	Layout storage — détection collisions

Outil	Type	Description
query_memory(query)	Mémoire	Interroge Cognee — local + collectif selon tier
simulate_call(sig, args)	Lecture	Simulation Tenderly fork (optionnel)
anchor_finding(hash, meta)	Onchain	KeeperHub MCP — ancre pendant l'analyse

Le 7ème outil anchor_finding rend l'agent véritablement autonome. query_memory vérifie d'abord les known-findings — si le pattern est déjà documenté, l'agent ne le reporte pas.

Niveaux de confiance : SUSPECTED → LIKELY → CONFIRMED. Seuls LIKELY et CONFIRMED déclenchent un anchor.

Phase 6 — Format de rapport enrichi

Format inspiré de MockSystems. Chaque finding inclut une référence au prior audit et un code sketch de correction — les deux éléments qui rendent un rapport actionnable plutôt qu'informatif.

```
## [F-001] Reentrancy in withdraw()

Severity:            HIGH
File:                Vault.sol:142-167
Prior audit ref:     None  ← ou ID finding mémoire Cognee
Confidence:          CONFIRMED
Onchain proof:       0x7f2e... (KeeperHub)

Description
withdraw() calls external address before zeroing user balance.
Attacker can re-enter and drain funds.

Recommendation
Apply CEI pattern (Checks-Effects-Interactions):
  - balances[msg.sender] = 0;   // effect first
  - (bool ok,) = msg.sender.call{value: amount}('');  // then interact
```

5. KeeperHub + 0G — Architecture Onchain

KeeperHub est une plateforme d'automation blockchain no-code avec un MCP server, une Direct Execution API, et un wallet Para MPC auto-provisionné.

Repositionnement — le vrai rôle de l'onchain

L'utilité principale de l'onchain n'est pas le timestamp non-falsifiable des findings. C'est l'ancrage des contributions à la mémoire collective — ce qui la rend immuable, vérifiable, et décentralisée.

> **Sans onchain vs avec onchain**
> Sans onchain : la mémoire collective est notre base de données privée. On peut la modifier, la censurer, la supprimer. Les utilisateurs n'ont aucune garantie que leurs contributions sont intactes.Avec onchain + 0G : chaque contribution est anchrée. Ni nous ni personne ne peut la modifier rétroactivement. Si notre startup disparaît, la base reste accessible sur 0G. La mémoire collective n'appartient à personne — elle appartient à la communauté.

	KeeperHub	0G
Rôle	Notaire — ancre le hash	Coffre-fort — stocke le contenu
Ce qu'il contient	pattern_hash + merkle_root (32+32 bytes)	Pattern complet : snippet + description + fix
Propriété	Immuable onchain	Décentralisé permanent
Si on disparaît	Hash vérifiable à jamais	Contenu accessible à jamais

La correction n'est pas dans la tx

La fix suggestion n'est pas dans la transaction onchain — trop volumineuse et inutilement coûteuse en gas. Elle est dans le payload 0G lié au hash. La correction est authentifiée par le pattern_hash sans être onchain.

```
# Ce qu'on ancre onchain (KeeperHub) — minimal
{
  'pattern_hash':  '0xa3f8c2...',  # 32 bytes
  'merkle_root':   '0xd4e9a1...',  # 32 bytes
  'contributor':   '0xWallet...',  # 20 bytes
  # timestamp = block.timestamp — gratuit
}

# Ce qu'on stocke sur 0G (décentralisé) — complet
{
  'pattern_hash':         '0xa3f8c2...',  # lien avec onchain
  'normalized_snippet':   'call{value: VAR_AMOUNT}...',
  'abstract_description': 'external call before state update',
  'fix_pattern':          'Apply CEI: effect before interaction',
```

```
  'severity':              'HIGH',
  'confidence':            'CONFIRMED',
}
```

Le terminal affiche les deux opérations simultanément :

```
Finding CONFIRMED — anchoring + storing...
✓ Onchain proof:  0x7f2e... (KeeperHub)
✓ Pattern stored: 0G://bafkrei... (fix inclus)
```

0G Storage — intégration concrète

0G Storage est S3 décentralisé et permanent. On uploade un JSON, on reçoit un rootHash (Merkle root). Ce rootHash est l'adresse permanente du pattern sur le réseau 0G — récupérable par n'importe qui, même si notre serveur disparaît.

Wrapper Node (0g/0g_upload.js)

```
import { ZgFile, Indexer, MemData } from '@0gfoundation/0g-ts-sdk';
import { ethers } from 'ethers';

const RPC_URL     = 'https://evmrpc-testnet.0g.ai';
const INDEXER_RPC = 'https://indexer-storage-testnet-turbo.0g.ai';
const signer = new ethers.Wallet(process.env.OG_PRIVATE_KEY,
               new ethers.JsonRpcProvider(RPC_URL));
const indexer = new Indexer(INDEXER_RPC);

// Upload JSON → rootHash
const data = new TextEncoder().encode(process.argv[2]);
const memData = new MemData(data);
const [tree, _] = await memData.merkleTree();
const [tx, err]  = await indexer.upload(memData, RPC_URL, signer);
console.log(tree.rootHash());  // stdout → Python
```

Client Python (storage/zero_g_client.py)

```
import subprocess, json, tempfile

async def store_pattern(entry: dict) -> str:
    """Upload un pattern sur 0G. Retourne le rootHash."""
    payload = json.dumps(entry)
    result = subprocess.run(
        ['node', '0g/0g_upload.js', payload],
        capture_output=True, text=True, check=True
    )
    return result.stdout.strip()  # rootHash 0G

async def retrieve_pattern(root_hash: str) -> dict:
    """Récupère un pattern depuis 0G via son rootHash."""
    result = subprocess.run(
        ['node', '0g/0g_download.js', root_hash],
        capture_output=True, text=True, check=True
    )
    return json.loads(result.stdout)
```

Flow complet — contribution d'un pattern

```
# Quand l'agent confirme un finding et que l'user opt-in :

# 1. Stocker le pattern complet sur 0G
root_hash = await store_pattern({
    'pattern_hash':         finding.pattern_hash,
    'normalized_snippet':   finding.snippet,
    'abstract_description': finding.description,
    'fix_pattern':          finding.fix,    # ← fix inclus
    'severity':             finding.severity,
})

# 2. Ancrer le hash + rootHash 0G via KeeperHub
tx_hash = await anchor_via_keeperhub(
    pattern_hash=finding.pattern_hash,
    root_hash_0g=root_hash  # ← lien KeeperHub ↔ 0G
)

# 3. Output dans le terminal
# ✓ Pattern stored : 0G rootHash = 0x7f2e...
# ✓ Onchain proof  : tx_hash     = 0x3bc4...
```

Vérification indépendante

N'importe qui peut vérifier la chaîne complète : tx KeeperHub → rootHash 0G → pattern complet avec fix. Sans avoir besoin de nous.

```
1. Etherscan → tx KeeperHub → { pattern_hash, root_hash_0g }
2. 0G StorageScan → root_hash_0g → pattern JSON complet
3. SHA-256(pattern_hash) == pattern_hash anchoré → ✓ intègre
```

AnchorRegistry.sol — contrat mis à jour

Le contrat ancre maintenant le rootHash 0G avec le pattern_hash — les deux sont liés onchain.

```
contract AnchorRegistry {
    event Anchored(
        bytes32 indexed patternHash,
        bytes32 indexed rootHash0G,   // ← lien vers 0G Storage
        address indexed contributor,
        uint256 timestamp
    );
    function anchor(bytes32 pHash, bytes32 root0G) external {
        emit Anchored(pHash, root0G, msg.sender, block.timestamp);
    }
}
```

Para MPC Wallet — zéro gestion operator

Avantage vs operator wallet custom

Chaque compte KeeperHub inclut automatiquement un wallet Para MPC. Clé partagée entre KeeperHub et Para — aucune des deux parties ne peut signer seule. Zéro gestion de notre côté : on finance ce wallet en ETH une fois. Gas estimation, nonce management, retry — tout est géré automatiquement.

Option A — Direct Execution API (filet de sécurité)

Notre code Python appelle KeeperHub en Phase 5 pour vérifier et compléter les anchors manqués par l'agent.

```
# keeper/direct_api.py
async def anchor_contribution(pattern_hash: str, merkle_root: str) -> str:
    response = await httpx.post(
        'https://app.keeperhub.com/api/execute/contract-call',
        headers={'X-API-Key': KEEPERHUB_API_KEY},
        json={
            'contractAddress': ANCHOR_REGISTRY_ADDRESS,
            'network': 'base-sepolia',
            'functionName': 'anchor',
            'functionArgs': json.dumps([pattern_hash, merkle_root])
        }
    )
    status = await poll_execution(response.json()['executionId'])
    return status['transactionHash']
```

Option B — MCP depuis l'agent (démo juges)

L'agent décide lui-même d'ancrer pendant son raisonnement via le 7ème outil — ce qui rend l'agent véritablement autonome.

```
# keeper/mcp_tools.py
async def anchor_finding_mcp(pattern_hash: str, merkle_root: str) -> str:
    wallet = await mcp.call_tool('get_wallet_integration')
    result = await mcp.call_tool('execute_workflow', {
        'workflow_id': ANCHOR_WORKFLOW_ID,
        'input': {'hash': pattern_hash, 'root': merkle_root}
    })
    logs = await mcp.call_tool('get_execution_logs', {
        'execution_id': result['executionId']
    })
    return logs['transactionHash']
```

Stratégie combinée — ceintures + bretelles

	Option B — MCP agent	Option A — Direct Execution API
Qui appelle	Agent Claude (Phase 4)	Notre code Python (Phase 5)

	Option B — MCP agent	Option A — Direct Execution API
Quand	Au moment où l'agent confirme un finding	Après que l'agent a terminé
Rôle	Ancrage en temps réel — narratif autonome	Filet de sécurité — exhaustivité
Usage principal	Démo juges	Production — fiabilité garantie
Si ça échoue	Phase 5 prend le relais	Log warning, pattern stocké 0G sans proof

MCP tools confirmés (docs KeeperHub)

get_wallet_integration — walletId Para pour les write ops.execute_workflow — déclencher un workflow existant.get_execution_status — vérifier qu'une tx a atterri.get_execution_logs — récupérer le tx hash.

Workflow KeeperHub à créer (setup Jour 1)

Node	Type	Configuration
Trigger	Webhook	Reçoit { hash, root } depuis notre backend
Action	Write Contract	AnchorRegistry.anchor(hash, root) sur Base Sepolia

6. Mémoire Collective — Le Vrai Moat

Le problème fondamental de tout outil open source : un utilisateur peut forker le repo, ajouter ses propres clés API, et obtenir le même résultat gratuitement. x402 ne résout pas ça si on vend juste un wrapper Anthropic.

> **Le vrai moat**
> La valeur n'est pas dans le code — elle est dans la mémoire collective. Un utilisateur qui forke démarre avec une mémoire vide. Un utilisateur qui paie accède à une base de milliers de patterns de vulnérabilités indexés. L'agent est immédiatement plus précis, plus confiant, et upgrade des findings SUSPECTED en CONFIRMED dès le premier audit.

Mémoire locale vs Mémoire collective

	Mémoire locale (gratuite)	Mémoire collective (payante via x402)
Commande	keeper-memory audit ./src/ --local	keeper-memory audit ./src/
Coût	Gratuit	1 USDC via x402
Contenu	Tes propres audits uniquement	Tous les audits anonymisés de tous les users
Au départ	Vide — zéro patterns connus	Pré-peuplée avec hacks publics (Rekt.news)
Précision agent	SUSPECTED dominant	CONFIRMED fréquent dès le 1er audit
Fork possible ?	Oui — tu l'as localement	Non — la base collective reste chez nous

Ce qui est stocké — design anonymisé

On ne stocke jamais le code brut, le nom du projet, ni l'adresse du contrat. On stocke uniquement le pattern abstrait et anonymisé.

Ce qu'on NE stocke PAS

```
# Jamais dans la mémoire collective
contract_name    = 'Vault.sol'
contract_address = '0xDevAPreLaunch...'
project_name     = 'SuperDeFiProtocol'
raw_code         = 'function withdraw() { ... }'
```

Ce qu'on stocke — pattern normalisé

```
# CollectiveMemoryEntry
{
  pattern_hash:         'sha256(normalized_snippet)',  # irréversible
```

```
  pattern_type:          'reentrancy',
  normalized_snippet:    'call{value: VAR_AMOUNT}(ADDR_RECIPIENT)
                          MAPPING_USER[msg.sender][VAR_INDEX] = 0',
  abstract_description: 'external call before state update
                          in epoch-based vault withdrawal',
  contract_context:      'ERC20 vault',  # jamais le nom
  severity:              'HIGH',
  confidence:            'CONFIRMED',
  confirmation_count:    3,  # audits indépendants
}
```

La normalisation remplace tous les identifiants spécifiques par des placeholders génériques : yieldAmount → VAR_AMOUNT, feeRecipient → ADDR_RECIPIENT, epochId → VAR_INDEX. Le pattern est structurellement identique à des dizaines de projets connus — impossible de remonter à la source.

Anonymat — est-ce que la faille est exploitable ?

Pour exploiter une faille, un attaquant a besoin de 4 choses : savoir que le projet existe, connaître l'adresse du contrat déployé, comprendre la logique exacte, et avoir le capital. Notre mémoire collective ne fournit aucun de ces éléments.

Besoin de l'attaquant	Ce que notre mémoire donne	Exploitable ?
Nom du projet vulnérable	Rien — jamais stocké	Non
Adresse du contrat	Rien — jamais stockée	Non
Code exact de la faille	Pattern normalisé uniquement	Non
Logique métier spécifique	Description abstraite seulement	Non

Seul risque résiduel

Un attaquant qui est aussi utilisateur de Keeper Memory pourrait voir 'reentrancy in epoch-based vault' dans les résultats et chercher activement des projets correspondants onchain. Probabilité faible — quelqu'un qui cherche des failles à exploiter utilise Slither directement, pas un outil payant.

Modèle freemium — free tier vs paid tier

	Free tier (--local)	Paid tier (défaut)
Commande	keeper-memory audit ./src/ --local	keeper-memory audit ./src/
Paiement	Gratuit	1 USDC via x402
Slither + agent	Oui	Oui
Mémoire locale	Oui	Oui
Mémoire collective	Non	Oui — ~1800+ patterns
Findings CONFIRMED	Rares	Fréquents dès le 1er audit

	Free tier (--local)	Paid tier (défaut)
Fork donne accès à	Ce tier uniquement	Mémoire locale vide seulement

Système d'opt-in pour contribuer

Par défaut, tous les findings restent dans la mémoire locale. L'utilisateur choisit explicitement de contribuer à la mémoire collective.

```
keeper-memory audit ./src/

# Après l'analyse :
# 2 findings CONFIRMED. Contribute patterns to collective memory?
# This improves accuracy for all users. Patterns are anonymized.
# In exchange: earn 0.50 USDC credit for your next audit.
# [Y]es / [N]o
```

La contribution est rémunérée en crédits x402. Un utilisateur actif qui contribue régulièrement peut réduire significativement son coût par audit. C'est aussi l'argument pour 0G : la mémoire collective est stockée de façon décentralisée — personne ne peut la supprimer ou la contrôler.

Bootstrap — cold start résolu

Au lancement, on pré-peuple la mémoire collective avec tous les hacks publics documentés. Ces données sont publiques, légalement utilisables, et donnent immédiatement de la valeur.

Source	Contenu	Patterns estimés
Rekt.news	Top 100 hacks DeFi documentés	~500 patterns
Immunefi disclosures	Bug bounty reports publics	~300 patterns
Audit reports publics	OpenZeppelin, Trail of Bits, etc.	~1000 patterns
Total au lancement	—	~1800 patterns

7. Modèle de Paiement — x402

x402 est le standard HTTP-natif pour les paiements agents, lancé par Coinbase en mai 2025. Il active le code HTTP 402 "Payment Required" comme mécanisme de paiement réel en USDC.

La logique de facturation est alignée avec la valeur délivrée : le free tier (mémoire locale) est gratuit, le paid tier (mémoire collective) est tarifé dynamiquement selon la complexité du projet audité.

Ce que x402 paie exactement

x402 débloque l'accès à la mémoire collective pour un audit complet. Sans paiement, l'agent tourne en free tier avec la mémoire locale uniquement. Avec paiement, l'agent accède à ~1800+ patterns anonymisés stockés sur 0G — ce qui upgrade les findings SUSPECTED en CONFIRMED.Ce que x402 ne paie PAS : tokens Anthropic (clé API interne) ni gas KeeperHub (wallet Para). L'utilisateur paie pour la valeur de la base de connaissance collective décentralisée.

Pricing dynamique — par complexité

Prix flat rejeté : trop cher pour un petit contrat, pas assez pour un gros protocole. Le prix est calculé en Phase 0 selon le nombre de fichiers détectés, communiqué à l'utilisateur avant paiement.

```
# phase0_resolve.py — calcul du prix
def calculate_price(resolved: ResolvedContract) -> float:
    files = len(resolved.files)
    if files <= 3:   return 0.50  # petit contrat
    elif files <= 10: return 1.00  # projet standard
    elif files <= 30: return 2.00  # protocole moyen
    else:             return 4.00  # gros protocole
```

Scope	Prix	Exemple
≤ 3 fichiers	0.50 USDC	Token ERC20 simple
≤ 10 fichiers	1.00 USDC	Vault + Router + 3-4 interfaces
≤ 30 fichiers	2.00 USDC	Protocole DeFi complet
> 30 fichiers	4.00 USDC	Stack complexe type Uniswap v4

Confirmation avant paiement

```
keeper-memory audit ./src/

Analyzing scope...
→ 12 files detected (Vault.sol, Router.sol, ...)
→ Audit price: 1.00 USDC (collective memory)
→ Press Enter to pay, Ctrl+C to run free (--local)
```

```
[x402] Paying 1.00 USDC... ✓ tx: 0x8fa3...
[x402] Collective memory unlocked — 1847 patterns available
```

Deux routes serveur — free vs paid

```
# server.py
@app.post('/audit/local')
async def run_audit_local(path: str):
    return await pipeline.run(path, collective_memory=False)

@app.post('/audit')
async def run_audit_collective(request: Request, path: str):
    scope = await resolve_scope(path)
    price = calculate_price(scope)           # dynamique
    if not verify_x402_payment(request, amount=price):
        return x402_response(amount=price)  # 402 avec le bon montant
    return await pipeline.run(path, collective_memory=True)
```

Économie circulaire — contribution

Un utilisateur qui contribue des patterns à la mémoire collective reçoit des crédits x402. Plus il contribue, moins il paie.

```
# Après l'analyse — opt-in contribution
# 2 findings CONFIRMED. Contribute patterns to collective memory?
# Patterns are anonymized. In exchange: earn credits.
#   +1 pattern → -0.25 USDC sur le prochain audit
#   +3 patterns → prochain audit gratuit
# [Y]es / [N]o
```

Comportement terminal — free vs paid

Free tier

```
keeper-memory audit ./src/ --local
  query_memory('reentrancy') → 0 collective hits
  Finding: SUSPECTED (no memory context)
  Tip: unlock collective memory (+1800 patterns) → 0.50 USDC
```

Paid tier

```
keeper-memory audit ./src/
  query_memory('reentrancy') → 3 collective hits
  Memory hit: Euler Finance (2024-03-15) — $197M — confidence +2
  Finding upgraded: SUSPECTED → CONFIRMED
```

Modèle économique

Poste	Montant	Payeur
Accès mémoire collective (x402)	0.50–4.00 USDC	User — paid tier

Poste	Montant	Payeur
Crédit contribution opt-in	-0.25 USDC/pattern	Nous — si user contribue
Tokens Anthropic (haiku + sonnet)	~$0.50–1.20	Nous — clé API interne
Gas KeeperHub + 0G anchor	~$0.05–0.25	Nous — wallet Para
Free tier	Gratuit	Coût Anthropic absorbé (~$0.50 acquisition)

Hackathon : prix fixe 1 USDC, architecture dynamique en place mais non activée. La migration vers le pricing dynamique est une ligne de config.

8. Testing & Mode Dev

Pendant le hackathon, les paiements x402, l'accès à la mémoire collective, et l'ancrage onchain sont bypassés via le flag --dev. La mémoire collective est simulée avec les hacks publics pré-chargés localement.

Flag --dev — bypass total

```
keeper-memory audit ./src/ --dev
# → [DEV] Using local memory only (collective bypassed)
# → [DEV] Skipping x402 payment
# → [DEV] Skipping onchain anchoring
# Pipeline complet, zéro coût
```

En dev, on simule le paid tier avec les hacks publics pré-chargés localement dans Cognee. L'agent se comporte comme si la mémoire collective était disponible — sans paiement réel.

Variables d'environnement

```
# .env — jamais committé
ANTHROPIC_API_KEY=sk-ant-...
KEEPERHUB_API_KEY=keeper_...
ETHERSCAN_API_KEY=...
ANCHOR_REGISTRY_ADDRESS=0x...    # déployé Jour 1
ANCHOR_WORKFLOW_ID=wf_...        # créé dans KeeperHub Jour 1
OG_PRIVATE_KEY=0x...             # wallet 0G testnet Galileo
OG_RPC_URL=https://evmrpc-testnet.0g.ai
OG_INDEXER_RPC=https://indexer-storage-testnet-turbo.0g.ai
NAMESPACE_API_KEY=ns_...         # Namespace SDK — ENS subnames
```

Budget hackathon

Usage	Modèle	Coût estimé
~100 audits de test	haiku + sonnet	~$10–20
Démo live pitch	haiku + sonnet	~$1–2
Gas KeeperHub (Sepolia)	Para MPC wallet	Gratuit (faucet ETH)
x402 (Base Sepolia)	USDC testnet	Gratuit (faucet Circle)
Total hackathon	—	~$15–25 max Anthropic

Action Jour 1 — crédits sponsors

Discord ETHGlobal OpenAgents → #keeperhub-resources (API key KeeperHub gratuite) et #sponsor-credits (crédits Anthropic éventuels). Faucets : sepoliafaucet.com (ETH) + faucet.circle.com (USDC Base Sepolia).

Résumé des 4 modes

Mode	Commande	x402	Mémoire	Coût réel
Dev	--dev flag	Bypass	Locale + hacks publics simulés	~$15 max Anthropic
Free tier	--local	Aucun	Locale uniquement	Gratuit (Anthropic absorbé)
Paid tier testnet	défaut	USDC testnet (faucet)	Collective Sepolia	~$2 démo
Paid tier prod	défaut	1 USDC mainnet	Collective mainnet	Viable

9. Sponsors & Prizes

Sponsor	Track ciblé	Prize	Rôle dans le projet
KeeperHub	Focus Area 1 — Best Innovative Use	$5,000	Notaire onchain — ancre pattern_hash + rootHash_0G
0G	Best Autonomous Agents	$1,500	Storage décentralisé — patterns + fix sur testnet Galileo
ENS	Best ENS for AI Agents + Most Creative Use	$2,500 × 2	Registre de certification onchain : contrats audités reçoivent un subname vérifiable + réputation auditeurs
LangChain (optionnel)	Focus Area 2		Exposer anchor_finding comme tool réutilisable

0G — Track : Best Autonomous Agents

Notre agent correspond exactement à leur exemple dans les qualification requirements : "Advanced research/knowledge agent with persistent context on 0G Storage (KV for real-time state + Log for conversation/history) + self-fact-checking using verifiable 0G Compute inference".

On utilise 0G Storage KV — plus adapté que le file upload pour notre use case : lookup direct par pattern_hash en O(1), exactement l'usage prévu pour du real-time state.

```
# 0G KV Storage — plus propre que file upload
key   = pattern_hash          # SHA-256 du pattern
value = json.dumps(entry)     # snippet + fix + severity

# Écriture
await kv_set(STREAM_ID, pattern_hash, json.dumps(entry))
# Lecture directe
entry = await kv_get(STREAM_ID, pattern_hash)  # O(1)
```

Critère 0G	Notre implémentation
Au moins 1 composant 0G core	0G Storage KV — patterns JSON indexés par pattern_hash
Démo fonctionnelle sur testnet	Testnet Galileo — kv_set() + kv_get() en live demo
Agent avec contexte persistant	Mémoire collective d'agent de sécurité — fit direct
Verifiable permanence	rootHash anchoré onchain via KeeperHub — double vérifiabilité

ENS — Intégration : Certification Onchain + Réputation Auditeurs

ENS fait un travail structurel dans Keeper Memory. Deux registres publics décentralisés — vérifiables sans passer par notre backend.

Usage 1 — Registre de certification des contrats (primary)

Après chaque audit, le contrat audité reçoit un subname sous certified.keeper-memory.eth. Les text records contiennent le verdict, les counts, et les preuves liées (KeeperHub + 0G). Le dev affiche ce subname comme badge sur son GitHub, son README, son site.

```
# Après audit de Vault.sol à 0x7f2e...
vault-0x7f2e.certified.keeper-memory.eth
    text: verdict       = 'CERTIFIED'       # ou FINDINGS_FOUND
    text: high_count    = '0'
    text: medium_count  = '2'
    text: audit_date    = '2026-04-25'
    text: tx_proof      = '0x3bc4...'   # KeeperHub anchor
    text: report_hash   = '0xa3f8...'   # 0G Storage root
```

```
<!-- README.md du projet audité -->
## Security
Audited by Keeper Memory
vault-0x7f2e.certified.keeper-memory.eth
Verify: https://app.ens.domains/vault-0x7f2e.certified.keeper-memory.eth
```

La vérification est décentralisée. Elle fonctionne même si notre serveur est down. N'importe quel dev, investisseur ou intégrateur peut résoudre l'ENS en une ligne :

```
from ens import ENS
ns = ENS.from_web3(w3)
verdict = ns.get_text(
    'vault-0x7f2e.certified.keeper-memory.eth', 'verdict'
)
# → 'CERTIFIED' | 'FINDINGS_FOUND' | None (jamais audité)
```

> **Pourquoi c'est non-cosmétique**
> ENS est habituellement utilisé pour résoudre nom → adresse. Ici on l'utilise dans l'autre sens : adresse → statut de sécurité. C'est un registre public de certifications Solidity — vérifiable par n'importe qui, sans notre backend, permanent. Aucun outil d'audit existant ne fait ça.

Usage 2 — Réputation des auditeurs contributeurs

Quand un user contribue un pattern à la mémoire collective (opt-in), il reçoit un subname sous auditors.keeper-memory.eth. Sa réputation est portable dans tout l'écosystème Web3.

```
0x7f2e.auditors.keeper-memory.eth
    text: contributions      = '14'
    text: reputation_score   = '87'
    text: confirmed_findings = '9'
```

Implémentation — Namespace offchain SDK

```
# ens/subname_manager.py
async def mint_cert(contract_addr: str, report: AuditReport) -> str:
    label = f"contract-{contract_addr[:6].lower()}"
    await httpx.post(
```

```
        'https://api.namespace.ninja/api/v1/subnames',
        headers={'x-auth-token': NAMESPACE_API_KEY},
        json={
            'label':      label,
            'parentName': 'certified.keeper-memory.eth',
            'owner':      contract_addr,
            'textRecords': {
                'verdict':      report.verdict,
                'high_count':   str(report.high_count),
                'audit_date':   report.date,
                'tx_proof':     report.tx_hash,
                'report_hash':  report.report_hash,
            }
        }
    )
    return f"{label}.certified.keeper-memory.eth"
```

Output terminal — Phase 6

```
[Phase 6] Generating report...
  ✓ 0 HIGH findings — eligible for certification
  ✓ ENS cert minted: vault-0x7f2e.certified.keeper-memory.eth
    verdict     = CERTIFIED
    tx_proof    = 0x3bc4... (KeeperHub)
    report_hash = 0xa3f8... (0G Storage)

Share your badge:
  https://app.ens.domains/vault-0x7f2e.certified.keeper-memory.eth
```

Track ENS	Notre fit	Prize
Best ENS for AI Agents	L'agent Keeper Memory mint des certifications ENS de façon autonome après chaque audit. ENS = mécanisme de vérification d'identité onchain.	$1,250
Most Creative Use	ENS comme registre public de sécurité Solidity : adresse → statut de sécurité. Inverse du use case habituel (nom → adresse). Vérifiable sans backend.	$1,250

KeeperHub — Argument Focus Area 1

La plupart des équipes utiliseront KeeperHub pour automatiser des opérations DeFi classiques. On l'utilise pour rendre l'audit de sécurité Solidity non-répudiable et vérifiable onchain. L'agent ancre chaque finding pendant son propre raisonnement via le MCP. Le finding existe sur la blockchain avant même que quelqu'un lise le rapport.

10. LangChain — Décision

Plan d'action Jour 1

Tutorial officiel LangChain Agents (2h) : python.langchain.com/docs/tutorials/agents. Si comfortable → intégrer + viser les 2 Focus Areas KeeperHub. Sinon → Anthropic SDK + Focus Area 1. Le projet est solide sans LangChain.

11. Planning — 2 Semaines

Semaine 1 — Core

Jour	Objectif
J1	Déployer AnchorRegistry.sol sur Sepolia. Créer workflow KeeperHub. Tutorial LangChain (2h). Setup .env complet. Enregistrer keeper-memory.eth (testnet).
J2	Scaffold CLI + server FastAPI. x402 middleware. x402-py client. Premier audit qui paie et tourne end-to-end.
J3	Phase 0 upstream detection. Phase 1 known-findings dedup. Phase 2 Slither diff only. Gate risk_score.
J4	Phase 4 Agent sonnet — system prompt adversarial. 7 outils + query_memory avec dedup. SSE streaming terminal.
J5	0G Storage KV setup (testnet Galileo). kv_set() + kv_get() qui tournent. KeeperHub workflow avec rootHash_0G. Flow complet : find → store 0G KV → anchor KeeperHub → double proof.
J6 – J7	ENS subnames contributeurs via Namespace SDK. Cognee mémoire graphe. Bootstrap mémoire collective (Rekt.news). Tests cross-audits.

Semaine 2 — Polish & Demo

Jours	Objectif
J8 – J10	Tests sur vrais contrats hackés (Euler Finance, Ronin, Rekt.news). Validation mémoire + ENS subnames.
J11 – J12	Frontend Next.js (optionnel). Rapport PDF. UX streaming terminal peaufiné.
J13 – J14	README + diagrammes Mermaid. Démo enregistrée. Pitch final.

12. Prochaines Étapes

Avant le hackathon (24 avril)

- Spec complète du module Cognee — mémoire locale + collective
- Script de bootstrap : parser Rekt.news + audit reports publics → CollectiveMemoryEntry[]
- Scaffold Python complet — structure de fichiers, pyproject.toml, dépendances
- Setup 0G testnet Galileo — wallet + RPC endpoints + tester kv_set/kv_get
- Enregistrer keeper-memory.eth sur testnet ENS — pointer vers le wallet opérateur
- Setup Namespace API key — tester mint_auditor_subname()
- Tester x402-py sur Base Sepolia avec un wallet local

- Décision LangChain après tutorial Jour 1

Questions ouvertes

- x402 facilitateur Base Sepolia — vérifier disponibilité testnet (faucet.circle.com)
- Tenderly simulation (Phase 4) — optionnel, semaine 2 si temps disponible
- Trail of Bits skills repo — patterns de vulnérabilités curatés, intégration roadmap post-hackathon